Lab 2 Report for Computer Architecture

Bohan Cui
ID: 221220089
NJU, Department of computer science and technology
Nanjing, China
bohan.cui@outlook.com

I. Run

It is an **push-button** solution for this lab.

You can just set the environment variable GEM5_ROOT to your gem5 root directory, and follow the README in the lab code directory.

About the detailed instruction of setup environment and install prerequisite, please refer to the README.

II. Analysis

To make the report more logically coherent, I will change the order of the 3 given problems to (3, 2, 1).

Besides, the tracing will cause great pressure to the simulator and make it very slow, so I reduce the ITERATIONS to 10000.

The asm of the two groups is as below:

```
...
401750: 48 89 ca  mov %rcx,%rdx
401753: a9 01 00 00 00  test $0x1,%eax
401758: 48 0f 45 d6  cmovne %rsi,%rdx
40175c: 83 c0 01  add $0x1,%eax
40175f: 49 01 d0  add %rdx,%r8
401762: 3d 10 27 00 00  cmp $0x2710,%eax
401767: 75 e7  jne 401750 <main+0x20>
401769: 4c 89 c2  mov %r8,%rdx
...
```

Listing 1. cmov

```
...
401748: 48 89 ca  mov %rcx,%rdx
40174b: a9 01 00 00 00  test $0x1,%eax
401750: 74 07  je 401759 <main+0x29>
401752: 48 c7 c2 05 00 00 00  mov $0x5,%rdx
401759: 83 c0 01  add $0x1,%eax
40175c: 49 01 d0  add %rdx,%r8
40175f: 3d 10 27 00 00  cmp $0x2710,%eax
401764: 75 e2  jne 401748 <main+0x18>
401766: 4c 89 c2  mov %r8,%rdx
...
```

Listing 2. branch-over

A. Q3: Gem5 + localBP

```
ncmov-localbp:
Total 89301500 ticks, 84913 instructions, 164074 operations, 178604 cycles.
Among 22009 branches, 5740 are mispredicted, 59 are predicted taken incorrectly, 5681 are predicted not taken incorrectly.
cmov-localbp:
Total 59236500 ticks, 79914 instructions, 149075 operations, 118474 cycles.
Among 12009 branches, 738 are mispredicted, 59 are predicted taken incorrectly, 679 are predicted not taken incorrectly.
```

Listing 3. retrieve outputs

We can find cmov out-perform branch-over greatly. We can find branch-over has 10000 more branches (the 'je') and about 5000 more mispredictions than cmov and most of the mispredictions are not-taken. We can easily deduce that the gap of the performance comes from that the predictor predicts 50% the branch-over branches wrongly.

Then let us inspect the trace. We can find with cmov, the pipeline progress smoothly, so it takes fewer cycles. (Fig. 1)

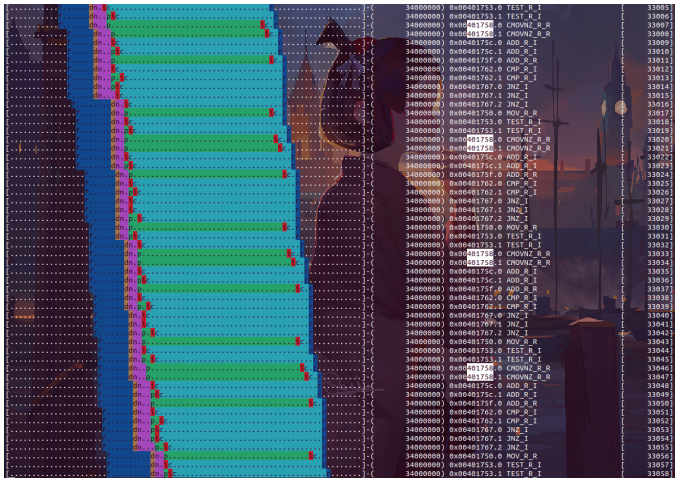

Fig. 1. trace of the cmov

When it comes to the branch-over, we can find the similar loops like in the Fig. 2. We can find that after several iterations, the outer loop is always predicted correctly (0x00401764). The branch-over branches (0x00401750, highlighted) will always be predicted **not-taken**. So the prediction will be wrong once in every two iterations. And due to the fact that the aggressive OoO pipeline will be flushed, it will cause significant penalty.('=' in the Fig. 2.). You can find that the penalty is even larger than 500%. Commonly in the trace, 84 uops will be flushed due to each wrong prediction. In comparison, every correct iteration only has 15 uops.

Obviously, it is because the 2-bit localBP will be stuck in $00 < -- > 01$ states when the branches are TNTNTNT-NTNT... (i & 1 for i in range(ITERATION)).

So this is a case that the branches are **poorly predicted**. It is natural that the cmov is the winner.

Fig. 2. trace of the branch-over

B. Q2: Gem5 + TournamentBP

Then let us try the case that the branch-over branches are well predicted. Using more powerful Tournament predictor, most branches are predicted correctly, as below.

```
1 ncmov-tournamentbp:
2 Total 56993500 ticks, 84913 instructions, 164074 operations,
      113988 cycles.
3 Among 22009 branches, 818 are mispredicted, 52 are predicted
      taken incorrectly, 766 are predicted not taken incorrectly.
4 cmov-tournamentbp:
5 Total 59554500 ticks, 79914 instructions, 149075 operations,
      119110 cycles.
6 Among 12009 branches, 810 are mispredicted, 51 are predicted
      taken incorrectly, 759 are predicted not taken incorrectly.
```

Listing 4. retrieve outputs

Branch-over and CMOV have similar mispredictions. When we check the trace, we can find both groups predict all the branches correctly pretty soon after enter the loop.

Then we should dive into the trace to find the reason why branch-over has fewer cycles given more instructions.

Let us take 'retire' as the mark of the pipeline throughput. According to Fig. 3, when the MOV is taken, branch-over takes 3 cycles to finish one iteration. And when the MOV is not taken(Fig. 4), branch-over only takes 2 cycles to finish one iteration. But when it comes to cmov, it always takes 3 cycles to finish one iteration(Fig. 5). Therefore, CMOV plan takes about 5000 more cycles.

So why? We can find that the cmov's instructions are 'piled-up' gradually.(Fig. 6) We can find out the key is in the three stuck-out rows in the figure. They are three dependencies

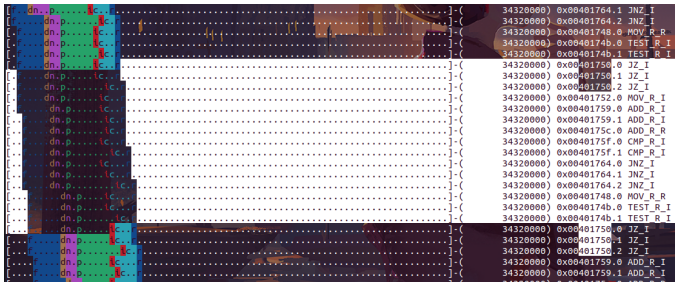

Fig. 3. trace for branch-over (MOV taken)

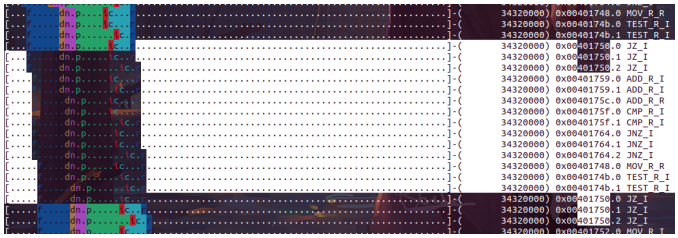

Fig. 4. trace for branch-over (MOV not taken)

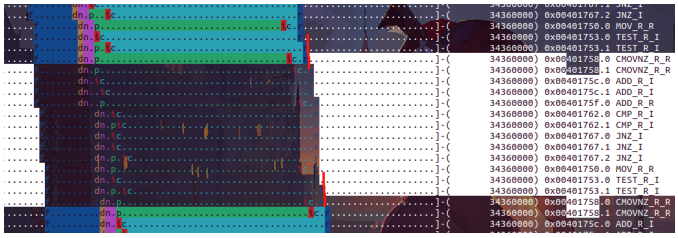

Fig. 5. trace for cmov

among [401750: 48 89 ca mov %rcx,%rdx], [401758: 48 0f 45 d6 cmovne %rsi,%rdx], [40175f: 49 01 d0 add %rdx,%r8]. (in Listing 1).

- add —-(%rdx, WAR)—- mov
- mov —-(%rdx, WAW)—- cmovne
- cmovne —-(%rdx, RAW)—- add(next iteration)

But in branch-over, there is no cmovne, and the cmp and je are not in this key path.

je is not needed to be run after other instrs, as shown in Fig. 7.

C. Q1: Native Run

The thing is becoming a bit strange when it comes to the native run.

Let us see the results first. I use the **perf** to track the performance. Due to the fact that the lab platform do not have perf and hard to use apt to install with root, so I run them on my laptop. Then something strange happened.

CPU:

Lab platform: Intel Xeon Gold 5118. (core: Skylake)

My laptop: Intel Core i5-11320H (core: Tigerlake)

When the tasks are compiled on the platform and run on the platform. As shown in **TABLE 1** (and some examples in Fig. 8 and Figure. 9), **branch-out is faster than cmov** when

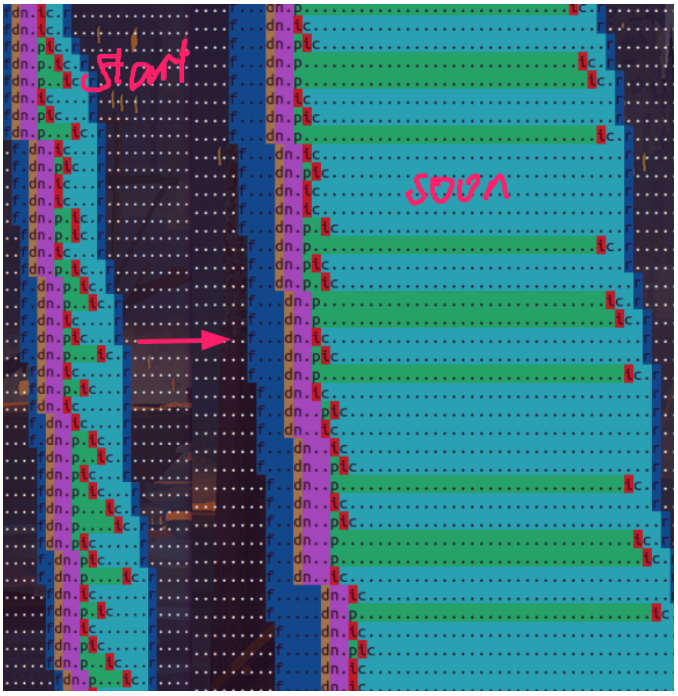

Fig. 6. piled up

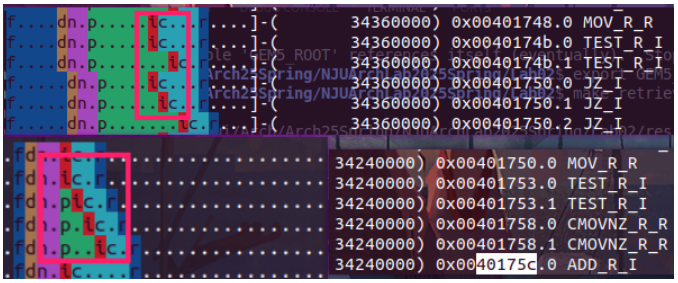

Fig. 7. Serialization (branch-over: up, cmov: down)

it is compiled on Xeon machine and run on the Core machine. When it is compiled on Core and run on Xeon machine, **cmov is twice as fast as branch-over**. When the compilation and run is on the same platform. **CMOV can outperform branch-over slightly**. This is **not the same as** Linus's result.

TABLE I
COMPILE AND RUN

compile \run	Xeon	Core
Xeon	ncmov: 0.817 cmov: 0.775	ncmov: 0.301 cmov: 0.420
Core	ncmov: 1.676 cmov:0.768	ncmov: 0.452 cmov: 0.437

It is very hard to gain an insight into this phenomenon, even if we do not take cross-platform into account. The execution details (prediction results , cycle-wise behavior) of the core are transparent to the system. What we can only know is coarse data in PMU. The perf results show that the branch prediction is successful and almost all of the branches are

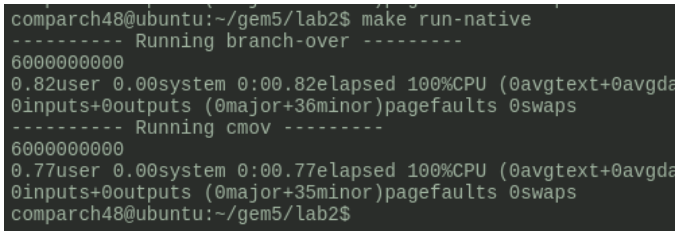

Fig. 8. Compilation: Xeon Run: Xeon

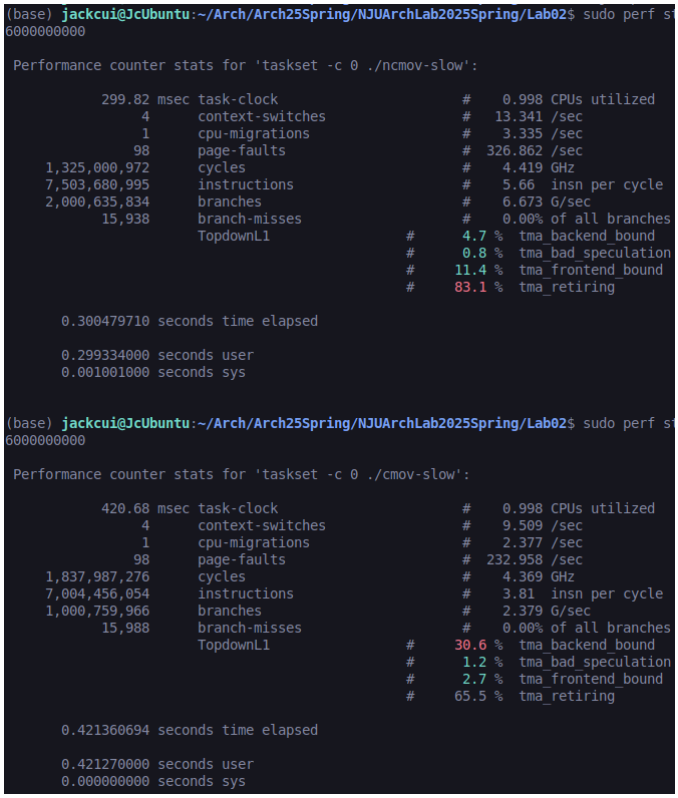

Fig. 9. Compilation: Xeon Run: Core

predicted correctly (quite normal on modern processors). And we can find that the asm of the main function is almost the same on the different platform.

So I can just provide two guesses for this phenomenon.

1. Modern CPU architecture (e.g. Skylake, Figure) may have more powerful OoO Engine, which can optimize the bottleneck like the cmov in the given task.

2. Different CPU models have different μop split rules. So dependencies can be a bit different on different platforms. However, the μop split rules is business secret of Intel. Few details are public. So it is a pity that we can not dive into this reason.

III. CUSTOMIZED PREDICTOR

Then let us implement a predictor. I design a **ReplayBP** based on the localBP. It try to find the patterns in the local loops. In addition to Saturated bits, ReplayBP records the last 20 actual branches. Predictor will try to find repetitions in the

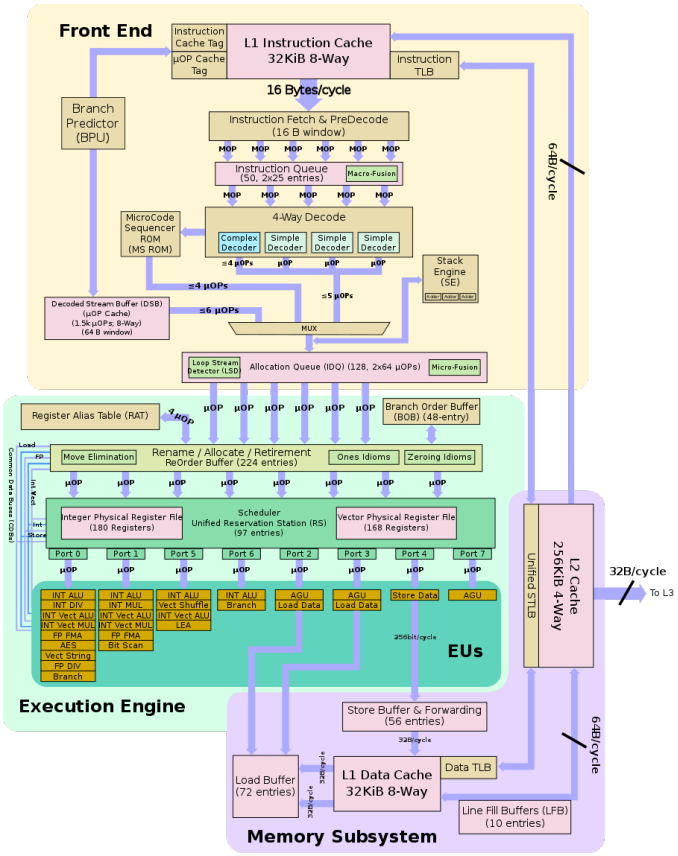

Fig. 10. Skylake

history. If there is a repetition in the history, it will predict the next according to the history. Otherwise, it will adopt the result of the 2-bit. You can see the full implementation in **replay.cc**.

```
1  if (firstFullCounter >= 20) {
2      for(int T = 5; T >= 1; T--) {
3          // find T-period repetition
4          for(int start = 19; start >= (10 / T) * T; start--) {
5          // at least repeat twice for period 4,5; 3 times for period
               3, 5 for 2, 10 for 1.
6              bool isRepetition = true;
7              int comparisonTargetIdx = start - T;
8              // check repetition
9              for (int i = comparisonTargetIdx; i >= 0; i -= T) {
10                 if (lastResult[i] != lastResult[start - ((start - i)
                       % T)]) {
11                     isRepetition = false;//
12                     break;
13                 }
14             }
15             if (isRepetition) {
16                 // return next prediction
17                 return lastResult[start - ((start - (-1) ) % T)];
18             }
19         }
20     }
21 }
```

Listing 5. core logic of ReplayBP

The ReplayBP runs well on the gem5! (Fig. 11)

The customization steps refer to

https://github.com/murattokez/gshare-gem5/blob/main/README.md

```
comparch48@ubuntu:~/gem5/lab2$ make run-new-pred
Gem5 is at /home/comparch48/gem5
Output target dir is /home/comparch48/gem5/lab2/res
Output source dir is /home/comparch48/gem5/m5out
---------- Running branch-over with ReplayBP ---------
/home/comparch48/gem5/build/X86/gem5.opt --outdir=/home/comparch48/gem5/
 --cmd=/home/comparch48/gem5/lab2/ncmov --cpu-type=DerivO3CPU --caches -
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 22.1.0.0
gem5 compiled May  2 2025 19:29:48
gem5 started May  2 2025 19:36:24
gem5 executing on ubuntu, pid 282093
command line: /home/comparch48/gem5/build/X86/gem5.opt --outdir=/home/co
/example/se.py --cmd=/home/comparch48/gem5/lab2/ncmov --cpu-type=DerivO3

warn: The `get_runtime_isa` function is deprecated. Please migrate away
warn: The `get_runtime_isa` function is deprecated. Please migrate away
Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot fi
build/X86/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mb
0: system.remote_gdb: listening for remote gdb on port 7003
**** REAL SIMULATION ****
build/X86/sim/simulate.cc:192: info: Entering event queue @ 0.  Starting
build/X86/arch/x86/cpuid.cc:180: warn: x86 cpuid family 0x0000: unimplem
build/X86/sim/mem_state.cc:443: info: Increasing stack size by one page.
build/X86/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
60000
Exiting @ tick 102495500 because exiting with last active thread context
```

Fig. 11. Run!

IV. Acknowledgement

Thanks for the guidance and help from the teacher and the teaching assistant!